TUI real-tools + visible-ensemble-members re-record — QA evidence (2026-06-14)

Run-id: `tui-ensemble-videos-20260614` **Operator request (2026-06-14):** *"record both videos now with models which do work with tooling so all our requests are resolved positive. question is — why ensemble behaves like it did, we have not seen multiple ensemble members responses in action!!!"*

Verdict: PASS (both videos, all 3 prompts, captured-frame evidence)

Both videos re-recorded by the agent driving the real standalone `helix-tui`, validated by extracting rendered frames and reading them (captured runtime evidence per §11.4.5 / §11.4.107 / §11.4.123 — no metadata-only PASS).

- **Video 1** — strongest single model (Groq `llama-3.3-70b-versatile`, picker digit 7): `~/Downloads/video1-strongest-model.mp4` (1.9 MB).
- **Video 2** — Helix Agent ensemble (picker digit 9): `~/Downloads/video2-helix-agent-ensemble.mp4` (3.9 MB, 268 s).

Captured frames (in `frames/`)

Frame	Proves
`video1-p3-git-status-status+log.png`	Single model autonomously calls `git_status` TWICE (default `status` + `subcommand=log`); real modified-files list + real commit log (`36bdee6a…`); coherent summary.
`video2-p1-codebase-3of4-members.png`	Ensemble panel renders **3/4 members** (Groq/Mistral/OpenRouter) each with a real codebase answer + score; `git_status` executed.
`video2-p2-agentsmd-4of4-members-shell-refused.png`	**4/4 members**; members used `grep`/`glob`/`git_status` to find the real `AGENTS.md`; **read-only safety guard visibly refuses** `tool: shell` → **"not permitted in read-only mode"** (§11.4.133).
`video2-p3-gitstatus-3of4-members.png`	**3/4 members** each give a real git-status summary; `git_status subcommand=branch` shows `+ main 36bdee6a [origin/main]`.

§11.4.138 bluff-audit — why the prior "Video 2 PASS 5/5" was a bluff

The 2026-06-13 Video 2 validation asserted only *"each prompt got a reply + no error tokens."* It did **not** assert (a) that multiple ensemble members were visible, nor (b) that any tool actually executed. The operator caught this manually ("we have not seen multiple ensemble members responses in action!!!"). Root causes found by systematic-debugging (§11.4.102) + LIVE reproduction:

1. **Members invisible by construction** — `EnsembleProvider.Generate` returned only the voted *winner's* text; every member's answer went into `ProviderMetadata`, which the TUI never rendered. → Added `FormatEnsemblePanel` (renders every participant + score + `[winner]`), wired into the TUI.
2. **No tool execution** — the TUI was a pure chat client (`GenerateStream`); it never used the existing `internal/agent`/`internal/tools` infra. → Added a reusable `agent.RunToolLoop` driver (multi-turn `Generate→tool-calls→execute→feed-back`) over a read-only-safe registry (`git_status`/`fs_read`/`glob`/`grep`), wired into the TUI submit path.
3. **Providers dropped tools on the wire** — Groq/DeepSeek/Mistral/OpenRouter (and the shared OpenAI types) did not send `tools`/`tool_choice` nor parse `tool_calls`; the tool-call CONVERSATION protocol (`assistant.tool_calls[].id` ⍰ `tool` message `tool_call_id`, `arguments` as a JSON **string**) was incomplete. → Completed across all 5 providers.
4. **Ensemble resolver rejected tool-call turns** — `generateMemberResilient` used a content-only success predicate, so a member's valid *empty-content + tool_calls* response was rejected and the resolver walked the catalogue to the decommissioned `gemma-7b-it`. → Predicate now `responseIsParticipant` (content OR tool-calls). LIVE-proven: ensemble returns a real `git_status` call with 3–4 members.
5. **Safety hole (caught in review)** — `NewToolRegistry(nil)` auto-registers write/shell tools and a nil approval manager **allows** them. → `RunToolLoop{ReadOnlyOnly:true}` offers + executes ONLY `LevelReadOnly` tools; the live video shows a `shell` call refused.

§11.4.135 permanent regression guards (all TDD, paired §1.1 mutation where noted)

- `internal/agent/tool_loop_test.go` — RunToolLoop multi-turn; **ReadOnlyOnly refuses a real write tool** (+ default-mode executes-it mutation guard).
- `internal/agent/tool_loop_protocol_test.go` — assistant(tool_calls) +tool(tool_call_id) pairing with matching ids.
- `internal/llm/tool_protocol_test.go` — 5-provider end-to-end wire protocol incl. `arguments` serialized as a STRING (mutation: object form → FAIL).
- `internal/llm/provider_tools_test.go` — per-provider tools send + tool_calls parse.
- `internal/llm/ensemble_provider_test.go::TestEnsemble_SentinelToolResolution_Accepts` — resolver accepts a tool-call turn (paired §1.1: content-only predicate reproduces the dead-model failure).
- `applications/terminal_ui/ensemble_render_test.go` — `FormatEnsemblePanel` asserts ≥2 members visible.
- `internal/tools/git/status_tool_test.go` — read-only allowlist; reject-test proves no mutation.

Reproduce

```
source /tmp/helix_keys.sh   # provider API keys (gitignored, never printed
cd helix_code && go build -o bin/helix-tui ./applications/terminal_ui/
vhs /tmp/v1_full.tape   # Video 1 (digit 7)
vhs /tmp/v2_full2.tape  # Video 2 (digit 9, 75s/prompt — ensemble is slow)
```

Sources verified 2026-06-14

- Groq tool-calling / decommissioned-model behavior: https://console.groq.com/docs/deprecations (observed live), OpenAI Chat Completions tool-call protocol (assistant `tool_calls` + `role:"tool"` `tool_call_id`, `arguments` as JSON string) — confirmed against live provider responses.